

- Guia de Deploy do Frontend (Next.js) no Ubuntu Server com Docker e Nginx
 - 1) Pré - requisitos
 - 2) Instalar Docker Engine e Docker Compose Plugin
 - 3) Preparar diretórios do projeto
 - 4) Obter o código do projeto
 - 5) Build e execução do frontend em Docker
 - 6) Instalar e configurar Nginx (proxy reverso)
 - 7) HTTPS com Let's Encrypt (Certbot)
 - 8) Firewall (UFW)
 - 9) Operações do dia a dia
 - 10) Notas específicas do projeto
 - 11) Solução de problemas

Guia de Deploy do Frontend (Next.js) no Ubuntu Server com Docker e Nginx

Este guia descreve, passo a passo, como implantar o frontend (Next.js) deste projeto em uma VM Ubuntu Server utilizando Docker, Docker Compose e Nginx como proxy reverso, incluindo HTTPS com Let's Encrypt.

Recomendado para Ubuntu Server 22.04 LTS ou 24.04 LTS com um usuário com privilégios de `sudo`.

1) Pré - requisitos

- Acesso à VM com usuário com `sudo`.
- Porta 80 (HTTP) e 443 (HTTPS) liberadas no provedor/cloud.
- Um domínio apontado para o IP público da VM (opcional, mas recomendado para HTTPS).
- Git instalado (ou outra estratégia para transferir os arquivos para o servidor).

```
sudo apt update && sudo apt install -y git ca-certificates curl gnupg lsb-release
```

2) Instalar Docker Engine e Docker Compose Plugin

Instalação via repositório oficial da Docker (método recomendado):

```
# Remover versões antigas (se houver)
sudo apt remove -y docker docker-engine docker.io containerd runc || true

# Configurar repositório oficial
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin

# Habilitar e verificar
sudo systemctl enable --now docker
docker --version
docker compose version

# (Opcional) Adicionar seu usuário ao grupo docker para não precisar de sudo
sudo usermod -aG docker "$USER"
newgrp docker
```

3) Preparar diretórios do projeto

Escolha um diretório para hospedar o projeto. Exemplo em `/opt/alfastore`:

```
sudo mkdir -p /opt/alfastore
sudo chown -R "$USER":"$USER" /opt/alfastore
cd /opt/alfastore
```

4) Obter o código do projeto

Existem duas opções comuns:

- Clonar via Git (substitua pela URL do seu repositório):

```
git clone <URL_DO_REPOSITORIO> .
```

- Transferir os arquivos do projeto (por SCP/SFTP/ZIP) para `/opt/alfastore`.

Certifique-se de que o arquivo `docker-compose.yml` está na raiz do projeto e que o diretório `frontend/` contém o `Dockerfile` e o `.dockerignore`.

Estrutura esperada (simplificada):

```
/opt/alfastore
├─ docker-compose.yml
└─ frontend/
    ├─ Dockerfile
    ├─ .dockerignore
    └─ package.json
```

5) Build e execução do frontend em Docker

No diretório raiz do projeto (`/opt/alfastore`), execute:

```
docker compose up -d --build
```

Isso irá:

- Construir a imagem do frontend (Next.js) em modo produção.
- Subir o container expondo a porta `3000` no host.

Valide localmente na VM:

```
curl -I http://127.0.0.1:3000
```

Se retornar **HTTP/1.1 200 OK** ou similar, o serviço está ativo localmente.

Observação: o **docker-compose.yml** já define **restart: unless-stopped**, para reiniciar o container automaticamente após reboot do host.

6) Instalar e configurar Nginx (proxy reverso)

Instale o Nginx no host:

```
sudo apt install -y nginx
sudo systemctl enable --now nginx
```

Crie o arquivo de configuração do site, por exemplo **/etc/nginx/sites-available/alfastore**:

```
sudo bash -c 'cat > /etc/nginx/sites-available/alfastore << "EOF"
server {
    listen 80;
    listen [::]:80;
    server_name SEU_DOMINIO_AQUI; # ex: loja.exemplo.com

    # Aumenta limite de upload se necessário
    client_max_body_size 10M;

    # Proxy para o frontend no Docker (porta 3000)
    location / {
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_pass http://127.0.0.1:3000;
    }

    # Cache agressivo para assets estáticos do Next
    location ^~ /_next/static/ {
        proxy_pass http://127.0.0.1:3000;
```

```
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    location ^~ /_next/image/ {
        proxy_pass http://127.0.0.1:3000;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }
}
EOF'
```

Habilite o site e verifique a configuração:

```
sudo ln -s /etc/nginx/sites-available/alfastore /etc/nginx/sites-enabled/alfastore
sudo nginx -t
sudo systemctl reload nginx
```

Neste momento, acessar http://SEU_DOMINIO_AQUI deve redirecionar as requisições ao container do frontend.

7) HTTPS com Let's Encrypt (Certbot)

Se tiver um domínio válido apontando para a VM:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d SEU_DOMINIO_AQUI
```

Siga o assistente para obter o certificado e, opcionalmente, forçar redirecionamento HTTP → HTTPS. Renovação automática é configurada pelo `certbot.timer`.

Testar renovação (simulação):

```
sudo certbot renew --dry-run
```

8) Firewall (UFW)

Se utilizar UFW, habilite apenas o necessário:

```
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw allow "Nginx Full"    # 80 e 443
sudo ufw enable
sudo ufw status
```

9) Operações do dia a dia

- Ver logs do frontend:

```
cd /opt/alfastore
docker compose logs -f frontend
```

- Reiniciar serviço do frontend:

```
docker compose restart frontend
```

- Atualizar código e reimplantar:

```
cd /opt/alfastore
# Se usa Git:
git pull
docker compose up -d --build
```

- Parar tudo:

```
docker compose down
```

10) Notas específicas do projeto

- O frontend é uma aplicação Next.js (ver `frontend/package.json`) e é servido em produção via `next start` na porta 3000 dentro do container.

- O arquivo `docker-compose.yml` na raiz já expõe a porta 3000 do container no host e define `restart: unless-stopped`.
 - O `next.config.ts` contém regras de `rewrites` para a API (já apontando para um backend público). Se precisar mudar a URL de backend, edite-o e reprocessse a imagem (`docker compose up -d --build`).
-

11) Solução de problemas

- Porta 3000 já usada no host:
 - Edite `docker-compose.yml` para usar outra porta, por exemplo `"8080:3000"`, e ajuste o `proxy_pass` do Nginx para `127.0.0.1:8080`.
 - Nginx não sobe ou retorna erro de configuração:
 - Verifique `sudo nginx -t` e logs em `/var/log/nginx/error.log`.
 - O site não abre externamente, mas abre via `curl 127.0.0.1:3000`:
 - Cheque DNS do domínio, regras de firewall/UFW, e se o Nginx está rodando e escutando em `0.0.0.0:80/443`.
 - Certbot falha ao emitir certificado:
 - Confirme que o domínio aponta para o IP público da VM e que a porta 80 está acessível externamente.
 - Após atualizar o código, mudanças não aparecem:
 - Reconstrua a imagem e suba novamente: `docker compose up -d --build`.
-

Pronto! Com esses passos, o frontend estará rodando em Docker e servido pelo Nginx com opção de HTTPS.